



Don't Be A Loser!

You don't know how much you don't know.

Read

electronics
FOR YOU

to keep up-to-date
with the tech world



To subscribe,
visit efy.in or
contact us at: 011-4059660
Email: support@efy.in

OR

Scan
This
Code



Also available at all leading magazine outlets

```
AI-Vision-Assistant-Camera-Code.ino
1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <Base64.h>
4  #include "esp_camera.h"
5  #include <Adafruit_GFX.h>
6  #include <Wire.h>
7  #include <Adafruit_SSD1306.h>
8  #include <ArduinoJson.h>
9  #include <U8g2lib.h>
10
11  const char* ssid = "<WIFI-SSID>";
12  const char* password = "<WIFI-PASSWORD>";
13  const String apiKey = "<OPENAI-API-KEY>";
14
15
16  // OLED display settings
17  #define SCREEN_WIDTH 128
18  #define SCREEN_HEIGHT 64
19  #define OLED_SCL 14
```

Fig. 5: Code snippet configuring the API key and Wi-Fi credentials

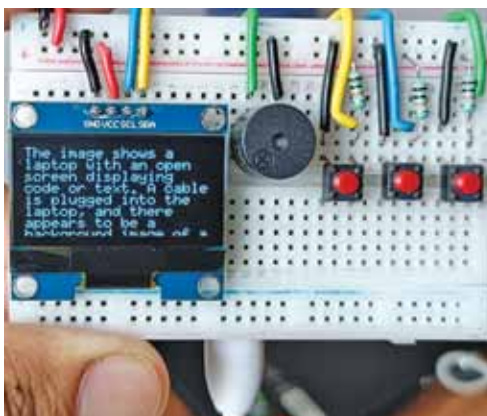


Fig. 6: Data shown on OLED during testing

To upload the source code into the ESP32-CAM, an external USB-to-FTDI adaptor is required, as the board does not have a built-in USB interface. Connect the FTDI adaptor's TX to U0R, RX to U0T, GND to GND, and 3.3V to 3.3V pins of the ESP32-CAM. Pull the IO0 pin to GND to enable flash mode, then upload the code using the Arduino IDE. After programming, disconnect

IO0 from GND and reset the ESP32-CAM to start running the uploaded code.

Construction and testing

Before assembling the components on the breadboard, as shown in the author's prototype (see Fig. 1), the source code must be uploaded to the ESP32-CAM. Then, the ESP32 camera should be connected to the FPC connector. Since most ESP32-CAM modules come with a pre-attached camera, this step can be skipped if it is already connected. Finally, the SSD1306 OLED display and buttons must be connected according to the circuit diagram.

After completing the assembly, the circuit can be powered up for testing. When a button is pressed, the processImage function captures an image using the ESP32-CAM module and converts it into Base64 format, as required by the OpenAI API. The analyse-WithAI function then prepares the payload with the encoded image and the selected prompt, while the makeAPIRequest function handles the HTTP communication with the OpenAI API.

Since AI responses often exceed the OLED display's capacity, an auto-scroll feature is implemented with a two-second delay between text segments. Once the full response is displayed, the system returns to the home screen, ready for the next operation. The three prompts can be customised to user needs, making the device versatile for both educational and accessibility applications. Fig. 6 shows the data displayed on the OLED during testing. **EFY**

TO DOWNLOAD SOURCE CODE AND RESOURCES



Scan this QR code or go to
<https://sourcecode.electronicshobby.com>
to access and download source code
and other resources for this project



Pushpendra Chandravanshi
is a software developer and
a robotics enthusiast.